

Key Differences Between Data Types and Data Structures

Aspect	Data Type	Data Structure
Definition	Specifies the type of data that can be stored in a variable.	Organizes data into collections to manage and access efficiently.
Category	Primitive or Reference data types.	Linear or Non-linear structures (e.g., Arrays, Linked Lists).
Examples	int, float, String, boolean	Array, Linked List, Stack, Queue, HashMap, Tree
Size	Defines the size of a single value (e.g., int is 4 bytes).	Defines how data is arranged and how much memory is used in storing a collection.
Purpose	To define the type of data a variable can hold.	To organize and store multiple data elements efficiently.

In summary:

- **Data types** are about the kind of data a variable can hold.
- **Data structures** are ways of organizing and storing multiple data elements.

1. Array in Java

An **array** in Java is an object that holds a fixed number of values of a single type. It is indexed and the size of an array is fixed once it is created.

Types of Indexing in Java

- **Zero-based indexing:** In Java, arrays are **zero-indexed**, meaning that the index of the first element is 0. The second element has an index of 1, the third element has an index of 2, and so on.

Example:

```
int[] arr = {10, 20, 30, 40};
```

```
// arr[0] = 10, arr[1] = 20, arr[2] = 30, arr[3] = 40
```

2. Array Initialization in Java

There are different ways to initialize arrays in Java. Here are the common methods:

- **Method 1: Declaration and Initialization at the same time**

```
int[] arr = {1, 2, 3, 4, 5}; // Initialize an array with values
```

- **Method 2: Declaration with size, then initialize each element**

```
int[] arr = new int[5]; // Array with 5 elements, each initialized to 0
```

```
arr[0] = 1; // Initialize array elements
```

```
arr[1] = 2;
```

```
arr[2] = 3;
```

```
arr[3] = 4;
```

```
arr[4] = 5;
```

- **Method 3: Using Arrays.fill() for initialization**

```
int[] arr = new int[5];
```

```
Arrays.fill(arr, 10); // Initialize all elements to 10
```

3. Insertion in Array in Java

Java arrays have a fixed size, meaning you cannot change their size after creation. Thus, **insertion** in the middle of an array isn't directly possible unless you create a new array to hold the elements after insertion.

Here's a simple example of how to insert an element in an array:

Insertion at the End of the Array (Simple Case)

If you want to insert an element at the end of the array and there's space, you can simply assign it like this:

```
public class Insertion {  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 30, 40}; // Initial array with 4 elements  
        int newElement = 50; // Element to be inserted  
  
        // Inserting at the end  
        int[] newArr = new int[arr.length + 1]; // New array with size 5  
        for (int i = 0; i < arr.length; i++) {
```

```

        newArr[i] = arr[i]; // Copying old array elements to new array
    }

    newArr[arr.length] = newElement; // Insert new element at the end


    // Printing the new array
    for (int i = 0; i < newArr.length; i++) {
        System.out.println(newArr[i]);
    }
}

```

Output:

Copy

10

20

30

40

50

Insertion at a Specific Position (Shifting Elements)

If you want to insert an element at a specific index, you need to shift the subsequent elements to the right to make room.

```

public class Insertion {

    public static void main(String[] args) {

        int[] arr = {10, 20, 30, 40}; // Initial array

        int position = 2; // Position where new element will be inserted

        int newElement = 25; // New element to insert
    }
}

```

```

// Inserting at position 2

int[] newArr = new int[arr.length + 1]; // New array with one extra element
for (int i = 0; i < position; i++) {
    newArr[i] = arr[i]; // Copy elements before the insertion point
}
newArr[position] = newElement; // Insert the new element
for (int i = position; i < arr.length; i++) {
    newArr[i + 1] = arr[i]; // Shift elements to the right
}

// Printing the new array
for (int i = 0; i < newArr.length; i++) {
    System.out.println(newArr[i]);
}
}
}

```

Output:

Copy

10

20

25

30

40

4. Deletion in Array in Java

Deletion in an array also requires creating a new array, as Java arrays are fixed in size. To delete an element, you need to shift the elements after the deleted item one position to the left.

Deletion of Element at a Specific Position

Here's how you can delete an element at a specific position:

```
public class Deletion {  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 30, 40}; // Initial array  
        int position = 2; // Position to delete (index 2 means element 30)  
  
        // Deleting at position 2  
        int[] newArr = new int[arr.length - 1]; // New array with one less element  
        for (int i = 0, j = 0; i < arr.length; i++) {  
            if (i != position) {  
                newArr[j++] = arr[i]; // Copy elements except the one to be deleted  
            }  
        }  
  
        // Printing the new array after deletion  
        for (int i = 0; i < newArr.length; i++) {  
            System.out.println(newArr[i]);  
        }  
    }  
}
```

Output:

Copy

10

20

40

In this example, we delete the element at index 2 (which is 30), and the rest of the elements are shifted to the left.

Summary

- **Array Initialization:** Arrays in Java can be initialized in several ways (direct initialization, using new, and using Arrays.fill()).
- **Indexing:** Arrays in Java are **zero-indexed**, meaning the first element is at index 0.
- **Insertion:** Insertion in an array involves creating a new array with a larger size, copying elements, and then inserting the new element.
- **Deletion:** Deletion involves creating a new array with a smaller size and shifting elements to fill the gap left by the removed element.

Finding Element in Array in lec

```
50      int key =60;
51      if(find(key))
52          System.out.print("Element found!" + key);
53      else
54          System.out.print("Element not found!" + key);
55
56      System.out.print("\n-----\n");
57
```

Insertion Method

```
//Insertion method:
static void insert(int value){
    if(n < a1.length){
        a1[n] = value;
        n++;
    }else{
        System.out.print("Array is full!");
    }
}
```

Deletion Element in array

```
//Deletion operation:
static void delete(int key){
    for(int i=0;i<n;i++){
        if(a1[i] == key){
            break;
        }
    }
}
```

Here is the **Time Complexity of Array Operations** in a **tabular form**:

Operation	Time Complexity	Explanation
Accessing an element by index	$O(1)$ (Constant time)	Direct access by index is a constant-time operation.
Inserting an element at the end	$O(1)$ (Amortized)	Inserting at the end is typically $O(1)$, but if resizing is needed (e.g., in dynamic arrays like ArrayList), it can take $O(n)$.
Inserting an element at a specific index	$O(n)$ (Linear time)	Insertion involves shifting all elements after the specified index.

Operation	Time Complexity	Explanation
Deleting an element	$O(n)$ (Linear time)	Deletion requires shifting all elements after the deleted element one position to the left.
Searching for an element (Unsorted)	$O(n)$ (Linear time)	A linear search examines each element of the array sequentially.
Searching for an element (Sorted)	$O(\log n)$ (Binary search)	Binary search works in logarithmic time for sorted arrays.
Updating an element	$O(1)$ (Constant time)	Updating an element at a specific index is a constant-time operation.
Resizing an array (Dynamic arrays)	$O(n)$ (Linear time)	Resizing involves creating a new array and copying all elements from the old array, which takes linear time.
Traversing the entire array	$O(n)$ (Linear time)	Iterating through all elements in the array takes linear time,

Operation	Time Complexity	Explanation
		where n is the number of elements in the array.

Key Points:

- **$O(1)$** : Constant time operations (e.g., accessing an element, updating an element).
- **$O(n)$** : Linear time operations (e.g., inserting, deleting, or traversing an array).
- **$O(\log n)$** : Logarithmic time (e.g., binary search on a sorted array).

This table summarizes the time complexity of common operations on arrays in Java (or in general programming).